COMPLETE BUILD GUIDE

LeakSense

IoT Gas Leak Detection & Pipeline Safety Platform

Hardware | Software | Integration | Architecture (Mermaid diagrams) | Budget | Build roadmap

Purpose. This guide is written so that a person who has never seen LeakSense can understand what it is, why each part exists, what to buy, how to wire it, how to run the software, how the two halves talk to each other, what it costs, and in what order to build and test it. Follow the sections in order.

How to read the labels in this guide
[Verified] = taken from the LeakSense System Master Guide (271/271 backend tests passing, frontend build clean). **[Recommended]** = a reference design added to make the guide buildable from scratch (pin map, packet format, wiring tips). Check it against your own code. **[Estimate]** = approximate price or time; confirm with current vendor quotes.

Safety and certification notice
LeakSense is a prototype and academic/pilot platform. Real combustible-gas installations are hazardous-area work: they need ATEX / IECEx (or local equivalent) certified sensors, enclosures and valves, plus a qualified safety engineer. Hobby parts (MQ-4, generic solenoids, bare PCBs) must never be the only protection on a live gas line.

Contents

1. What LeakSense is and why it exists 2
2. System architecture 3
3. Hardware: the sensor node 4
4. Software 6
5. Integration: how hardware and software work together 8
6. Safety logic and alert levels [Verified] 10
7. Build roadmap from scratch 11
8. Budget [Estimate] 12
9. Calibration, troubleshooting and deployment 13
10. Known gaps, assumptions and next steps 14

1. What LeakSense is and why it exists

Pipelines carrying combustible gas (mainly methane, CH_4) fail in two ways: slowly (flange or gasket creep, small pressure loss) and suddenly (rupture). Legacy fixed alarms often fire only near the lower explosive limit, and a human closing a valve takes minutes. Cheap analog sensors also drift, freeze or spike, so plants either miss leaks or suffer false shutdowns.

Problem	LeakSense answer
Micro-leaks stay under fixed alarm thresholds	Rate-of-change analysis (pressure decay + gas creep) warns early
Manual valve closure is slow	ESP32 closes the solenoid valve itself in under 1.2 s; the server also commands CLOSE
Sensors drift, freeze, spike	Watchdogs: flat-line (5 identical readings), spike (CH_4 jump ≥ 100%), out-of-range, node silence
Alerts get missed	Siren, in-app bell, live WebSocket banner, SMS (critical), email, weekly PDF audit
No accountability	Audit trails for every valve action and every threshold change

Project goals and scope

- A working engineering prototype: 1 to 4 sensor nodes covering monitoring zones A to D (up to six zones supported by the design)
- A business-ready B2B product concept: industrial gas users, first target Malaysia, then Singapore and Indonesia
- Out of scope for the prototype: certified hazardous-area hardware, multi-site cloud tenancy, ML models (listed as future work)

Key numbers [Verified]

Item	Value
Edge valve closure	< 1.2 s after a physical limit breach
Server telemetry interval	1 reading per node every 60 s over Wi-Fi
Methane critical trigger	350 ppm (editable by admin)
Pressure collapse trigger	$\Delta P \geq 15\%$
Detection range	0 to 10,000 ppm CH_4
Node silence watchdog	180 s (3 missed 60 s cycles)
Valve command acknowledge window	120 s, then TIMEOUT alert
Login token life	24 hours (JWT)
Test status	271 / 271 backend tests passing

2. System architecture

LeakSense has four tiers. Data flows up (sensor packets, readings, alerts) and commands flow down (valve close/open, alarm silence).

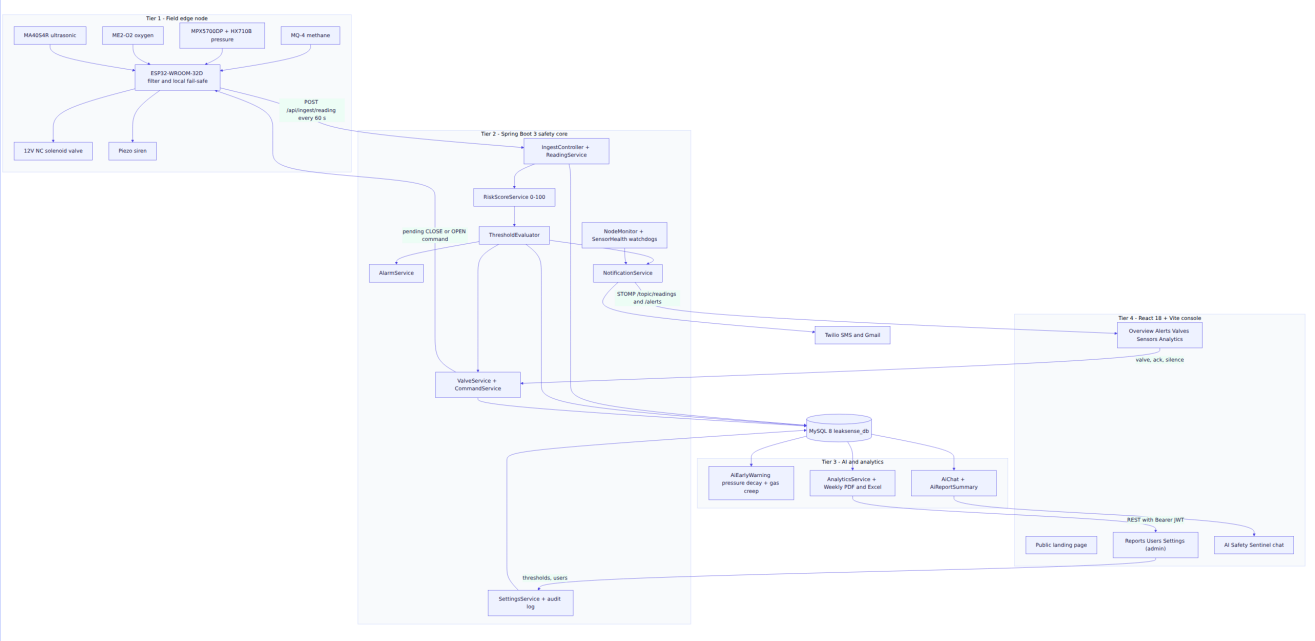


Figure 2.1 - Full system architecture (Mermaid). Edge node, Spring Boot safety core, MySQL, AI/analytics and the React console; arrows show data and command flow.

Why this design

- **Edge decides first.** The ESP32 enforces a local limit so the valve closes even if Wi-Fi or the server is down.
- **Server decides smarter.** It adds history, risk scoring, sensor-health logic and operator control.
- **Database is the audit spine.** Readings, alerts, alarms, valve actions and threshold changes are all stored.
- **Dashboard is read-mostly.** Operators watch, acknowledge, silence and control valves; admins also manage users and thresholds.

Technology stack [Verified]

Layer	Technology
Edge	ESP32-WROOM-32D (Arduino/ESP-IDF), Wi-Fi 2.4 GHz, optional LoRa SX1278
Backend	Java 21, Spring Boot 3, Spring Security 6, Spring Data JPA/Hibernate, STOMP WebSocket, OpenPDF, Apache POI, Twilio SDK, Gmail SMTP
Database	MySQL 8.x (tests use in-memory H2)
Frontend	React 18, Vite, Axios, Recharts, STOMP client, CSS variables for dark/light theme
Quality	JUnit 5, Mockito, AssertJ (27 test classes), ESLint 9
Dev tools	IntelliJ IDEA, VS Code, Wokwi (ESP32 simulation), Postman, Git/GitHub

3. Hardware: the sensor node

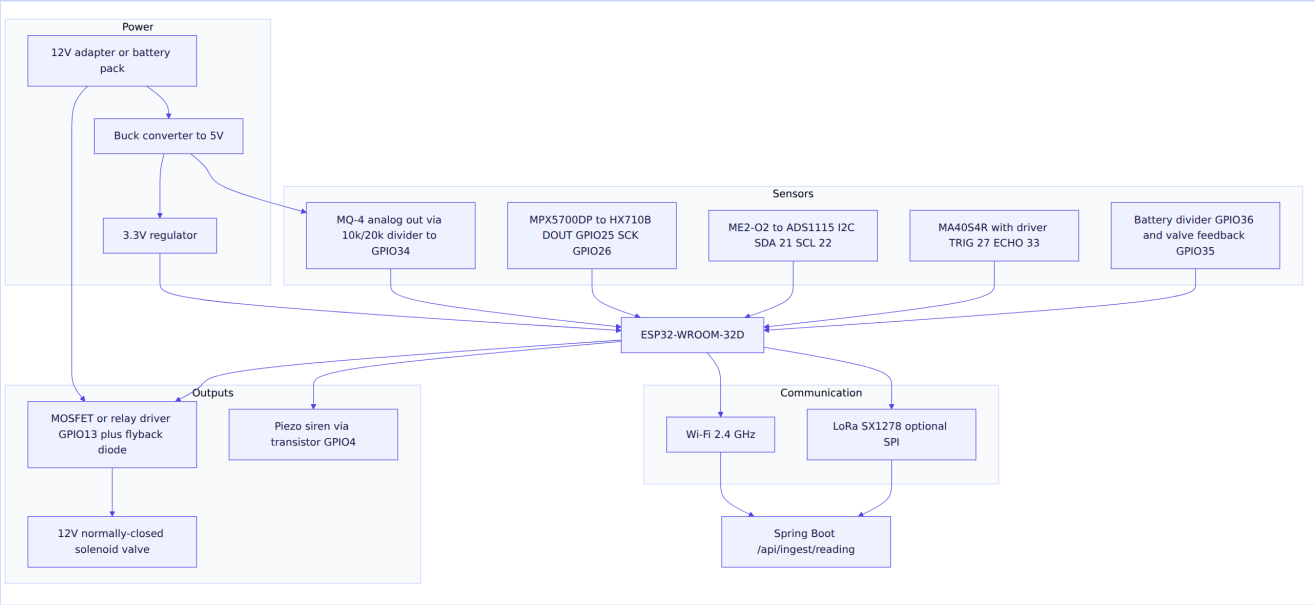


Figure 3.1 - Hardware block and power (Mermaid), with the recommended ESP32 pins. Normally-closed valve = fail-safe on power loss.

3.1 Components and what each does [Verified]

Part	Model	Range / spec	Role in the system
Combustible gas	MQ-4 semiconductor	200 to 10,000 ppm CH_4	Methane leak detection; drives the 350 ppm valve trigger
Pressure	MPX5700DP + HX710B ADC	0 to 700 kPa (0 to 100 psi)	Line integrity; flags drops ≥ 15%
Oxygen	ME2-O2 electrochemical	0 to 25% vol	Detects oxygen displacement; alarm area below 19.5%
Ultrasonic	MA40S4R	40 kHz transducer	Auxiliary sensing at the node
Controller	ESP32-WROOM-32D	240 MHz dual core, 2.4 GHz Wi-Fi	Reads sensors, filters, local fail-safe, sends packets
Long range (optional)	LoRa SX1278	Sub-GHz radio	Link where Wi-Fi does not reach
Actuator	12 V DC direct-acting solenoid valve	Up to 1.6 MPa, IP67	Physical line shutoff in under 1.2 s
Alarm	Piezo siren	Local audible	Sounds on hazard; silenced from the dashboard

3.2 Wiring reference [Recommended]

Use ADC1 pins (GPIO32 to 39) for analog inputs, because ADC2 cannot be used while Wi-Fi is active. Change the mapping if your sketch differs, but keep one rule: **never connect a 5 V signal directly to an ESP32 pin.**

Signal	ESP32 pin	Notes
MQ-4 analog out	GPIO34 (ADC1)	Module output can reach 5 V: use 10 kΩ over 20 kΩ divider to stay ≤ 3.3 V
HX710B DOUT / SCK	GPIO25 / GPIO26	Powered at 3.3 V or 5 V per module; check the pressure signal sits within the HX710B input range
ME2-O2 via ADS1115	SDA GPIO21, SCL GPIO22	Sensor output is only tens of millivolts, so it needs the ADS1115 with high gain

Signal	ESP32 pin	Notes
Ultrasonic TRIG / ECHO	GPIO27 / GPIO33	The bare MA40S4R needs a driver/receiver circuit, or use a ready module; divide ECHO to 3.3 V
Valve driver input	GPIO13	N-MOSFET or relay board; flyback diode across the coil; separate 12 V rail
Piezo siren	GPIO4	Through a transistor if the siren draws more than 20 mA
Valve position feedback	GPIO35	Reed/limit switch; needed to detect a stuck valve
Battery voltage	GPIO36	Resistor divider; feeds the battery_level field
LoRa SX1278 (SPI)	SCK 18, MISO 19, MOSI 23, NSS 5, RST 14, DIO0 2	Optional; powered from 3.3 V only

3.3 Design rules that matter

- **Use a normally-closed (NC) valve.** When power is lost the valve closes: fail-safe.
- **Separate power rails.** 12 V for the valve, buck converter to 5 V for sensors, 3.3 V regulator for ESP32. Join all grounds at one point.
- **MQ-4 needs warm-up.** Leave it powered for several minutes before reading, and 24 hours burn-in the first time. Humidity and temperature shift its readings.
- **Generic hardware valves are not gas-rated.** Use a water solenoid only for bench demos; use a certified gas shutoff valve for any real gas.
- **Enclosure.** IP65 or better box, cable glands, fuse on the 12 V feed, and strain relief on every cable.
- **Battery option.** For off-grid nodes, size the pack for the 1-minute Wi-Fi duty cycle plus MQ-4 heater draw, which is the largest continuous load.

3.4 Firmware behaviour [Recommended]

```
setup(): init sensors, ADS1115, HX710B, valve driver (valve CLOSED at boot), siren off,
         connect Wi-Fi, load node code + device API key

every 1 s  (local safety loop - works even with no Wi-Fi):
    read CH4, pressure, O2  -> median/average filter
    if CH4 >= local_limit or pressure_drop >= 15% or O2 < 19.5%:
        close valve (<1.2 s), sound siren, set hazard_lock = true
    if sensor value out of physical range: mark sensor FAULT (do not trigger a trip)

every 60 s (telemetry):
    POST packet to /api/ingest/reading with device API key
    read response: pending commands (CLOSE / OPEN / SILENCE)
    execute command, send acknowledgement (server marks TIMEOUT after 120 s)

rules:  never OPEN while hazard_lock is set; OPEN only on a valid server command
        reconnect Wi-Fi with back-off; keep running local loop while offline
```

4. Software

4.1 Backend (Spring Boot, port 8080) [Verified]

Service	Responsibility
ReadingService	Receives packets, marks node ONLINE, refreshes lastSeen, clamps battery, flags out-of-range values, stores reading, pushes to /topic/readings
RiskScoreService	0 to 100 score = min(100, $CH_4/CH_4crit \times 60 + |\Delta P|/Pbaseline \times 25 + \max(0, 20.9 - O_2)/5 \times 15$)
ThresholdEvaluator	Compares readings with warning/critical limits; raises CRITICAL; asks ValveService to CLOSE when auto-close is on
ValveService	Open/close/emergency-close with role rules: only ADMIN may OPEN, opening is blocked during an active hazard
CommandService	Creates DeviceCommand records, waits for ESP32 acknowledgement, TIMEOUT after 120 s
NodeMonitorService	Scheduled watchdog: silent node (180 s) becomes OFFLINE with a DEVICE alert; flat-line detection
SensorHealthService	Spike ($CH_4 \geq 100\%$, pressure drop $\geq 15\%$, O_2 drop $\geq 10\%$) and 5-cycle freeze detection
AlarmService	Siren lifecycle ACTIVE, SILENCED, CLEARED; 5-minute silence re-sound timer
NotificationService / SmsService	In-app bell, STOMP push, email, Twilio SMS for CRITICAL (simulated fallback when no credentials)
SettingsService	Editable thresholds; every change logged old to new in ThresholdAuditLog
Weekly / PdfReportService	Monday 09:00 five-section PDF with charts and tables; Excel export
AI services	Early warning, chat with live DB awareness, 0-100 safety summary, insights
AnalyticsService	Trend series, risk buckets, alert timeline, incidents by sensor

4.2 Database [Verified]

MySQL database `leaksense_db`; Hibernate creates the tables on first start. Entities: Zone, SensorNode, Reading, Alert, AlarmEvent, ValveAction, DeviceCommand, AppNotification, AppUser, ThresholdSetting, ThresholdAuditLog. A Zone has many nodes, alerts and valve actions; a node has many readings.

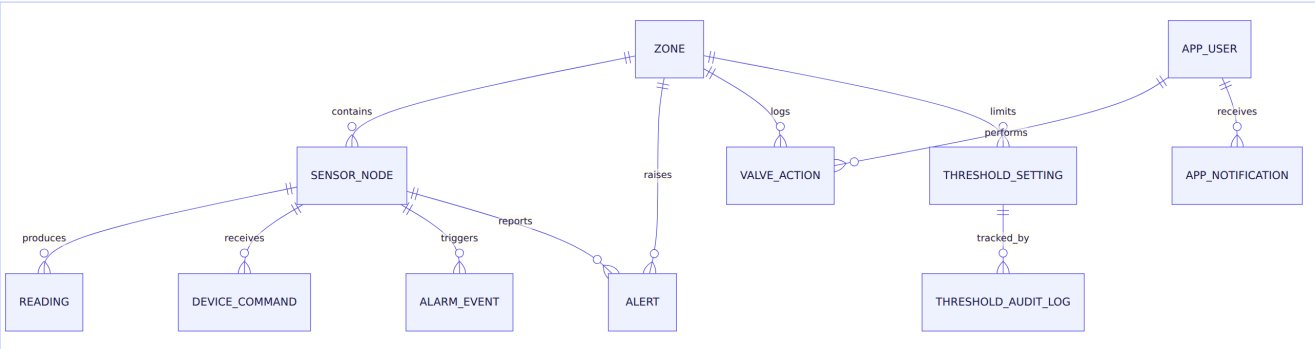


Figure 4.1 - Database entity relationships (Mermaid).

4.3 Security [Verified]

- Stateless JWT (HMAC-SHA256, 24 h); passwords stored as BCrypt hashes
- Public routes: `/api/auth/**`, `/api/public/**`, `/ws/**`, `/api/ingest/**` (ingest is protected by the device API key)
- OPERATOR: telemetry, alerts, alarms, valve actions. ADMIN: also users and thresholds
- Secrets live outside Git: `application-local.properties` (git-ignored) or environment variables: `SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD`, `LEAKSENSE_JWT_SECRET`, `LEAKSENSE_DEVICE_API_KEY`, `MAIL_USERNAME`, `MAIL_PASSWORD`

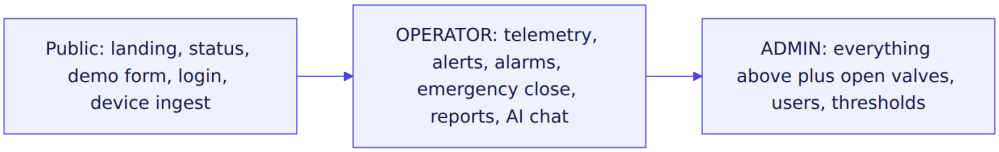


Figure 4.2 - Access levels: public, operator, admin (Mermaid).

4.4 API summary [Verified]

Group	Endpoints	Access
Auth	POST /api/auth/login	Public
Public site	GET /api/public/landing, /status; POST /demo-request	Public
Ingest	POST /api/ingest/reading	Device key
Readings	GET /api/readings/live, /history	Operator
Alerts	GET /api/alerts/active, /history; PUT {id}/ack, {id}/resolve	Operator
Alarms	GET /api/alarms/active; PUT {id}/silence, {id}/clear	Operator
Valves	GET /api/valves; POST {zoneId}/open, {zoneId}/close, /emergency-close-all	Operator (open: admin)
Analytics	/api/analytics/sensor-trends, risk-distribution, alerts-timeline, alerts-by-sensor, valve-activity	Operator
Reports and AI	/api/reports/weekly/pdf, /excel; /api/ai/chat, /report-summary	Operator
Notifications	GET /api/notifications, /unread-count; PUT {id}/read, /mark-all-read	Operator
Admin	/api/settings/thresholds (GET, PUT); /api/users (GET, POST, DELETE)	Admin

4.5 Frontend (React + Vite, port 5173) [Verified]

Page	What the user does there
Landing (public)	Overview of the product, hardware table, live heartbeat from /api/public/status, demo request form
Login	Operator / Admin sign in; session lives in sessionStorage so closing the tab logs out
Overview	KPI cards, alarm banner with Silence, active alerts (Acknowledge / Resolve), sensor health, valve audit trail, live gas chart
Analytics	Live packet stream and historical charts (trends, risk distribution, alert timeline, incidents by sensor)
Valves	Per-zone Open / Close with confirmation modal, Emergency Close, audit trail
Alerts	Filter by zone, severity, status, date; acknowledge and resolve
Sensors	Inventory, battery, last seen, Send Ping for testing
Reports	AI executive summary, PDF and Excel download, email the weekly report
Users / Settings	Admin only: accounts, thresholds, auto-shutoff toggles
AI Safety Sentinel	Floating chat assistant answering from live system state

4.6 Run it locally [Verified]

```
# prerequisites: JDK 21, Node.js 18+, MySQL 8 on port 3306
CREATE DATABASE IF NOT EXISTS leaksense_db;

cd leaksense_backend
./mvnw spring-boot:run        # http://localhost:8080  (seeds admin, operator, zones, nodes)
./mvnw test                   # 271 tests

cd leaksense-frontend
npm install
npm run dev                   # http://localhost:5173
npm run lint && npm run build
```

Default dev logins: `admin / admin123` and `operator / operator123`. These are for local development only; change them before any shared or real deployment.

5. Integration: how hardware and software work together

5.1 End-to-end data flow [Verified]

#	Step	Where	Result
1	Sense	ESP32 samples MQ-4, pressure, O_2	Filtered values
2	Local decision	ESP32	Valve closes in under 1.2 s and siren sounds on a limit breach
3	Transmit	ESP32 to POST /api/ingest/reading	Packet every 60 s
4	Ingest	ReadingService	Node ONLINE, reading saved, pushed to /topic/readings
5	Score and evaluate	RiskScoreService, ThresholdEvaluator	Risk 0 to 100; CRITICAL alert if $CH_4 \geq 350$ ppm or $\Delta P \geq 15\%$
6	Act	ValveService, CommandService, AlarmService	CLOSE command created, siren event opened
7	Health checks	SensorHealth, NodeMonitor	Spike, freeze, silence faults
8	Notify	NotificationService	Bell, WebSocket banner, SMS (critical), email
9	Operate	Dashboard	Operator acknowledges, silences, resolves, reopens (admin)
10	Audit	ValveAction, ThresholdAuditLog, weekly PDF	Permanent trail

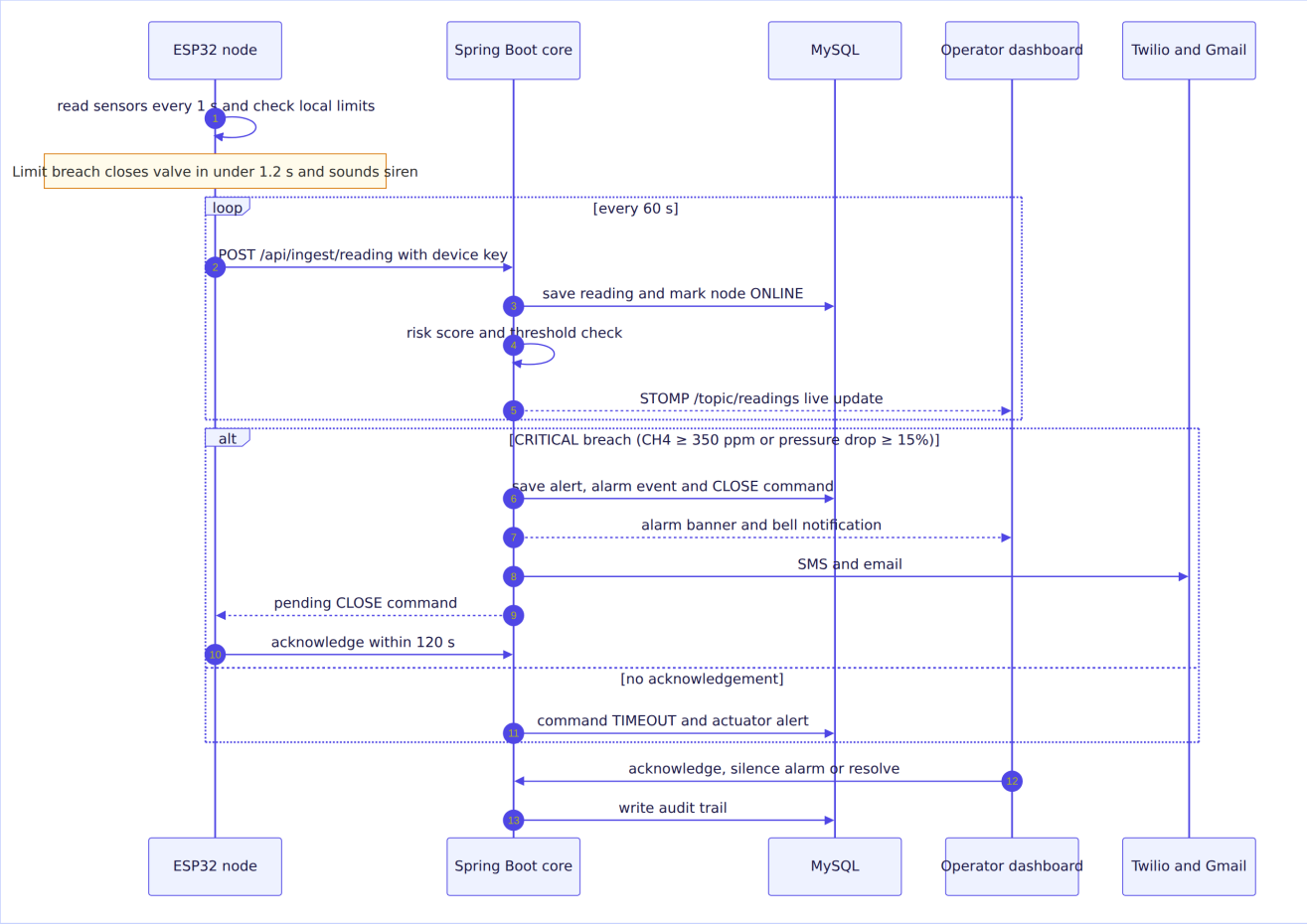


Figure 5.1 - One reading from sensor to dashboard, SMS and audit trail (Mermaid sequence diagram).

5.2 Telemetry packet [Recommended]

Field names follow the database columns; match them to the exact DTO in your `IngestController` and the header name used for the device API key.

```
POST /api/ingest/reading
Header: <device API key header>: <LEAKSENSE_DEVICE_API_KEY>
{
  "nodeCode":      "NODE-A1",
  "methanePpm":    182.4,
  "pressureKpa":   412.7,
  "oxygenPercent": 20.8,
  "batteryLevel":  87,
  "valveStatus":   "OPEN"
}
```

5.3 Timing you must understand [Verified]

Path	Speed	Why
ESP32 local trip	< 1.2 s	Runs on the node; independent of Wi-Fi and server
Server-driven CLOSE	Up to about 60 s	Server reacts when the next packet arrives (1-minute interval)
Stale-node alarm	180 s	Three missed intervals, avoids flapping
Command acknowledge	120 s	After that the command is marked TIMEOUT and an actuator alert is raised

5.4 Integration test plan

Test	How	Expected result
Ingest works	POST a packet with curl/Postman or the Sensors page Send Ping	Reading in Live Stream; node ONLINE
Threshold trip	Send CH_4 = 400 ppm	CRITICAL alert, siren event, CLOSE command, bell notification
Hazard lock	Try to OPEN the valve during an open critical alert	Request blocked
Role rules	Log in as OPERATOR and try to OPEN	Denied; admin allowed
Offline node	Stop sending for 3+ minutes	Node OFFLINE, DEVICE alert; auto-resolves when data returns
Frozen sensor	Send an identical value 5 times	Sensor failure flagged
Command timeout	Do not acknowledge a CLOSE	TIMEOUT after 120 s, actuator alert
Silence alarm	Press Silence on the banner	Alarm SILENCED; re-sounds after 5 minutes if hazard persists
Weekly report	Trigger the report from Reports page	PDF with five sections downloads
Real hardware	Repeat the above using the ESP32 and a safe test gas source in a ventilated area	Same behaviour as simulation

6. Safety logic and alert levels [Verified]

Check	Trigger	Outcome
Methane critical	CH_4 ≥ 350 ppm	CRITICAL alert, auto valve CLOSE (if enabled), siren
Pressure collapse	ΔP ≥ 15%	CRITICAL alert
Methane spike	Jump ≥ 100% vs previous	SENSOR_FAULT (suspected sensor, not a real leak)
Oxygen displacement	Drop ≥ 10% or O_2 below 19.5%	Hazard alert
Flat-line	5 identical readings	Sensor failure flagged
Silent node	No data for 180 s	OFFLINE + DEVICE alert
Stuck valve	No acknowledgement in 120 s	TIMEOUT + critical actuator alert
Out of physical range	Value outside sensor span	FAULT, kept separate from real hazards

Alert levels are warning and critical with high and low limits per sensor, plus sensor-failure ranges, all editable by an admin on the Settings page and checked every minute. SMS is sent for CRITICAL only; valve-closed and threshold-change events also notify.

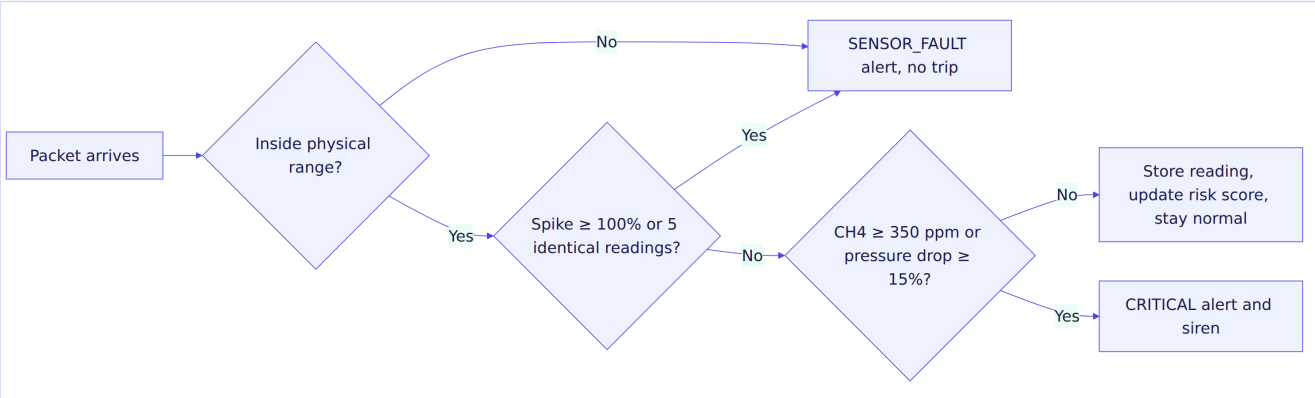


Figure 6.1 - Detection stage: what happens to every packet (Mermaid).

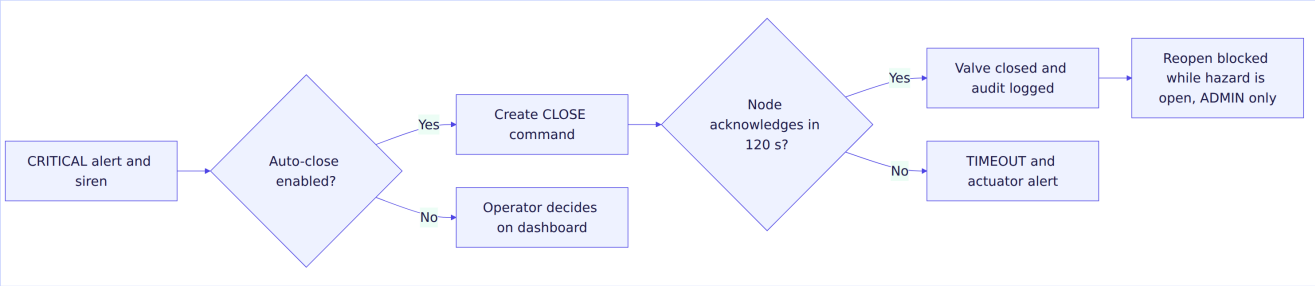


Figure 6.2 - Response stage after a CRITICAL breach (Mermaid).

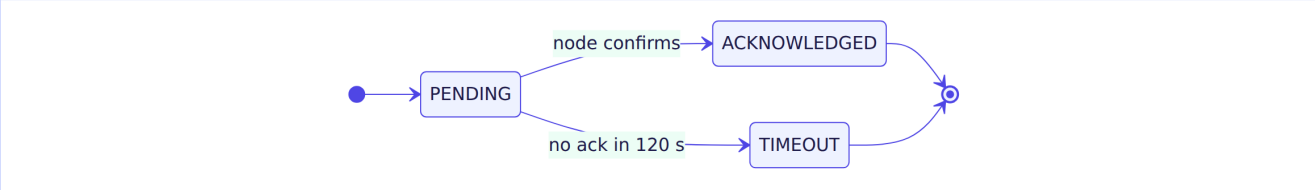


Figure 6.3 - Device command lifecycle (Mermaid state diagram).

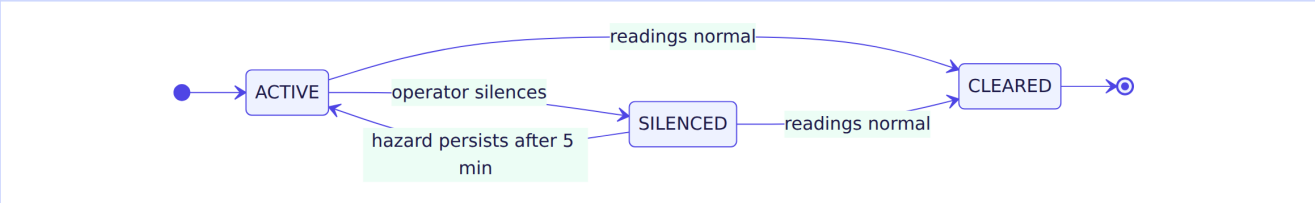


Figure 6.4 - Alarm lifecycle with the 5-minute silence timer (Mermaid state diagram).

7. Build roadmap from scratch

Order matters: finish and test the software first, then simulate the hardware, then build the real node. Times are [Estimate] for one motivated student working part-time.

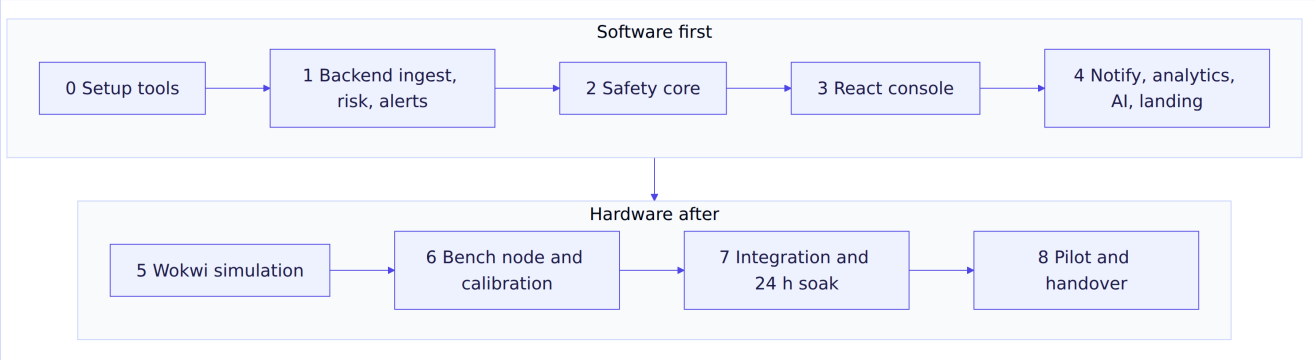


Figure 7.1 - Build roadmap: software first, then simulation and real hardware (Mermaid).

Phase	Goal	Key steps	Done when	Time
0 Setup	Tools ready	Install JDK 21, Node 18+, MySQL 8, IntelliJ, VS Code, Postman, Git; create two repos	Hello-world Spring Boot and Vite apps run	2 days
1 Backend core	Ingest to alert pipeline	Entities and repositories, ReadingService, risk score, ThresholdEvaluator, ingest endpoint with device key, JWT login	Postman packet creates a reading and a CRITICAL alert	2 to 3 weeks
2 Safety core	Watchdogs and valves	NodeMonitor, SensorHealth, ValveService, CommandService, AlarmService, settings + audit log	All safety rules covered by unit tests	2 weeks
3 Frontend	Operator console	Login, sidebar layout, Overview, Valves, Alerts, Sensors, Settings, Users, theme and mobile layout	Dashboard updates live over WebSocket	3 weeks
4 Extras	Notifications, analytics, AI	Bell, email, Twilio, analytics charts, weekly PDF/Excel, AI chat and summary, landing page	Every page in section 4.5 works	2 weeks

Phase	Goal	Key steps	Done when	Time
5 Simulation	Prove the edge logic	Build the ESP32 sketch in Wokwi with sliders for sensors; point it at the local backend	Sim node trips the valve and the dashboard shows it	1 week
6 Bench hardware	Real ESP32 node	Buy parts, breadboard, calibrate each sensor, add valve driver and siren	Node posts real readings every 60 s	2 to 3 weeks
7 Integration	Close the loop	Run all tests in section 5.4 with real hardware; tune thresholds; log a 24-hour soak test	No missed packets, correct trips	1 to 2 weeks
8 Pilot	Field readiness	Enclosure, power, certified valve review, GitHub release, documentation	Demo-ready; handover checklist complete	2 weeks

8. Budget [Estimate]

Prices are approximate Indian-market ranges for hobby/prototype parts in INR and change often. Treat them as planning numbers, then request live quotes. The generic valve line is for a bench demo; a certified gas shutoff valve costs far more.

Item (per node)	Qty	Low (INR)	High (INR)
ESP32-WROOM-32D dev board	1	450	700
MQ-4 methane sensor module	1	150	350
MPX5700DP pressure sensor	1	2,500	4,500
HX710B ADC module	1	100	250
ME2-O2 oxygen sensor	1	1,500	2,800
ADS1115 16-bit ADC module	1	300	500
MA40S4R ultrasonic transducer + driver circuit	1	200	500
12 V NC solenoid valve (bench demo grade)	1	400	1,000
MOSFET / relay driver module + flyback diode	1	100	250
Piezo siren / buzzer	1	50	300
Power: 12 V adapter or battery pack, BMS, LM2596 buck	1	900	2,000
IP65 enclosure, glands, fuse	1	350	800
Wires, connectors, resistors, PCB/perfboard	1	300	700
One-node subtotal		**7,300**	**14,650**

Shared and optional items	Low (INR)	High (INR)
LoRa SX1278 module pair (optional)	600	1,100
Multimeter, soldering kit, jumper set (if you do not own them)	1,200	3,000
Safe test gas source for response checks (small butane lighter refill, ventilated area, no flame)	100	300
Certified calibration gas and regulator for production-grade calibration (supplier quote)	5,000	15,000

Pilot scenarios

Scenario	Low (INR)	High (INR)
Prototype: 1 node + basic tools	8,600	17,950
Pilot: 4 nodes (zones A to D) + tools	30,500	61,900
Pilot with LoRa on all 4 nodes	32,900	66,300

Software and running costs [Estimate]

Item	Cost	Notes
IntelliJ Community, VS Code, MySQL, Postman, Git, Wokwi (free tier)	0	All free for this project
Gmail SMTP for email alerts	0	Use an app password, stored in env vars
Twilio SMS (critical alerts)	Pay per SMS	Check current India/target-country rates; trial credit is enough for testing
Cloud VPS for demos (backend + MySQL)	500 to 1,500 / month	Optional; local hosting works for the prototype
Domain name	about 800 / year	Optional, for the public landing page

Budget advice
The pressure sensor and oxygen cell dominate the cost. Build and test the whole software stack and the Wokwi simulation first (cost: nothing), then buy hardware for **one** node, prove it, and only then multiply for more zones.

9. Calibration, troubleshooting and deployment

Calibration checklist [Recommended]

- **MQ-4:** burn in 24 h, record clean-air baseline, check response with a small safe gas source in a ventilated area, map the analog reading to ppm with a curve; re-check after humidity or temperature changes
- **MPX5700DP:** read atmospheric pressure at rest and record the offset; verify against a reference gauge at two points
- **ME2-O2:** calibrate in fresh air to 20.9%; the cell drifts slowly and ages, so plan replacement
- **Valve:** time the close with a stopwatch or logic analyser; confirm the feedback switch reports the true position

Troubleshooting

Symptom	Likely cause	Fix
Node never appears ONLINE	Wrong API key, node code or backend URL	Check headers and node record; test with Postman first
Analog readings jump randomly	Using an ADC2 pin with Wi-Fi, or noisy supply	Move to ADC1 pins, add 0.1 µF capacitors, average samples
ESP32 resets when valve fires	Coil inrush through shared rail	Separate 12 V rail, flyback diode, bulk capacitor
False CRITICAL alerts	MQ-4 not warmed up or humidity drift	Wait for warm-up, recalibrate, raise filtering
Dashboard not live	WebSocket blocked or wrong URL	Check /ws path, CORS, browser console
Login asked again in a new tab or after closing the tab	Token is in sessionStorage by design	Sign in again; expected behaviour
Tests fail on a fresh machine	Missing JDK 21 or profile config	Use JDK 21; tests use the H2 profile
No SMS	Twilio credentials absent	System falls back to simulated mode; add credentials to env vars

Deployment and handover checklist

- Rotate seed passwords, generate a long random JWT secret and device API key
- Move every secret to environment variables; never commit `application-local.properties`
- Restrict CORS to the real frontend origin and serve over HTTPS
- Back up the MySQL database on a schedule
- Run a 24-hour soak test and a deliberate power-loss test (valve must close)

- Document zone map, node codes, threshold values and who receives SMS
- Have certified hardware and a qualified engineer approve any real gas installation

10. Known gaps, assumptions and next steps

- **Hardware is not field-tested yet** [Verified]; every hardware wiring detail here is a recommended design.
- **Documentation inconsistency to fix:** the master guide says the stale-node check uses 60 s in one place and 180 s in another. This guide uses 180 s.
- **Oxygen part name:** earlier project notes mention MIX8410 while the master guide specifies ME2-O2. Confirm which cell you purchase and update the document.
- **Ultrasonic sensor role** is described only as auxiliary; decide whether it measures pipe-room leak noise, tank level or distance and document it.
- **Valve command channel:** the guide says commands are acknowledged by the node; confirm in code whether the node polls or receives commands in the ingest response.
- **Future work:** LoRaWAN and cellular fallback, mobile push (FCM/APNs), 3D digital twin (Three.js), LSTM early-warning model via ONNX, hardware certification path, multi-site SaaS tenancy.

Glossary

Term	Meaning
CH_4 / ppm	Methane / parts per million
LEL	Lower explosive limit, the lowest gas concentration that can ignite
ΔP	Change in line pressure
NC valve	Normally closed: closes when power is removed
STOMP	Messaging protocol used over WebSocket for live dashboard updates
JWT	Signed login token sent as a Bearer header
RBAC	Role-based access control (OPERATOR vs ADMIN)
SCADA	Supervisory control and data acquisition, the operator console style
ATEX / IECEx	Certification schemes for equipment in explosive atmospheres